

AWS Three-Tier Architecture Deployment

Project Overview

This project demonstrates the manual implementation of a highly available Three-Tier Architecture on AWS using multiple Availability Zones. The architecture separates the application into Presentation Layer (Frontend), Application Layer (Backend), and Database Layer (RDS) to improve scalability, security, and fault tolerance.

Introduction

Three-Tier Architecture is a software design pattern that separates an application into three logical and physical layers:

1. Presentation Tier
2. Application Tier
3. Database Tier

This architecture improves security, scalability, maintainability, and availability.

Objective

To deploy a highly available and secure web application using AWS cloud services while separating frontend, backend, and database resources.

Architecture Diagram

Components

Presentation Layer

- Application Load Balancer (ALB)
- Frontend EC2 Instances
- Auto Scaling Group
- Private Web Subnets

Application Layer

- Internal Application Load Balancer
- Backend EC2 Instances
- Auto Scaling Group
- Private Application Subnets

Database Layer

- Amazon RDS MySQL
- Private Database Subnets
- Multi-AZ Deployment

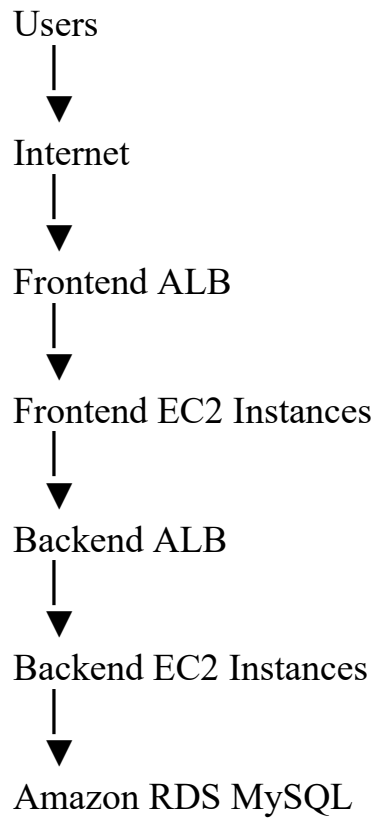
AWS Services Used

- Amazon VPC
- Public and Private Subnets
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups
- Application Load Balancer (ALB)
- EC2 Instances
- Auto Scaling Groups
- Launch Templates
- Amazon RDS MySQL
- IAM Roles

Network Design

Resource	CIDR
VPC	10.0.0.0/16
Public Web Subnet A	10.0.0.0/20
Public Web Subnet B	10.0.16.0/20
Public Web Subnet C	10.0.32.0/20
Private Web Subnet A	10.0.48.0/20
Private Web Subnet B	10.0.64.0/20
Private Web Subnet C	10.0.80.0/20
Private App Subnet A	10.0.96.0/20
Private App Subnet B	10.0.112.0/20
Private App Subnet C	10.0.128.0/20
Private DB Subnet A	10.0.144.0/20
Private DB Subnet B	10.0.160.0/20
Private DB Subnet C	10.0.176.0/20

Architecture Flow



Implementation Steps

Step 1: Create VPC

Create a VPC with CIDR block:

10.0.0.0/16

Step 2: Create Subnets

Create:

Public Subnets

Public-Web-Subnet-A

Public-Web-Subnet-B

Public-Web-Subnet-C

Private Web Subnets

Private-Web-Subnet-A
Private-Web-Subnet-B
Private-Web-Subnet-C

Private App Subnets

Private-App-Subnet-A
Private-App-Subnet-B
Private-App-Subnet-C

Private DB Subnets

Private-DB-Subnet-A
Private-DB-Subnet-B
Private-DB-Subnet-C

Step 3: Create Internet Gateway

Attach Internet Gateway to VPC.

Step 4: Create NAT Gateway

Create NAT Gateway in Public Subnet A.

Allocate Elastic IP.

Step 5: Configure Route Tables

Public Route Table

0.0.0.0/0 → Internet Gateway

Associate with Public Subnets.

Private Route Table

0.0.0.0/0 → NAT Gateway

Associate with Private Subnets.

Step 6: Create Security Groups

Frontend ALB SG

Allow:

HTTP 80
HTTPS 443

From: 0.0.0.0/0

Frontend EC2 SG

Allow:

HTTP 80

From Frontend ALB SG.

Backend ALB SG

Allow:

HTTP 80

From Frontend EC2 SG.

Backend EC2 SG

Allow:

HTTP 80

From Backend ALB SG.

Database SG

Allow:

TCP 3306

From Backend EC2 SG.

Step 7: Create Database Subnet Group

Include:

Private-DB-Subnet-A
Private-DB-Subnet-B
Private-DB-Subnet-C

Step 8: Launch Amazon RDS MySQL

Configuration:

Engine: MySQL

Class: db.t3.micro

Storage: 20 GB

Public Access: Disabled

Multi-AZ: Enabled

Step 9: Create Frontend EC2

Install:

```
sudo yum update -y  
sudo yum install git httpd -y
```

```
sudo systemctl start httpd  
sudo systemctl enable httpd
```

```
cd /var/www/html
```

```
sudo git clone <github-repository>
```

Step 10: Create Backend EC2

Install:

```
sudo yum update -y  
sudo yum install git httpd php php-mysqlnd -y
```

```
sudo systemctl start httpd  
sudo systemctl enable httpd
```

Deploy backend application.

Configure database connection.

Step 11: Create Frontend Target Group

Frontend-TG

Register Frontend Instances.

Step 12: Create Backend Target Group

Backend-TG

Register Backend Instances.

Step 13: Create Frontend ALB

Deploy ALB in Public Subnets.

Attach Frontend Target Group.

Step 14: Create Backend ALB

Deploy Internal ALB in Private App Subnets.

Attach Backend Target Group.

Step 15: Create Launch Templates

Create:

Frontend Launch Template

Backend Launch Template

Step 16: Create Auto Scaling Groups

Frontend ASG

Min: 2

Desired: 2

Max: 4

Backend ASG

Min: 2

Desired: 2

Max: 4

Testing

Frontend

Open:

`http://frontend-alb-dns-name`

Verify application loads.

Backend

Verify API communication.

Database

`mysql -h <rds-endpoint> -u admin -p`

Security Best Practices

- Database placed in private subnets.
- RDS not publicly accessible.
- Security Group based access.
- Multi-AZ deployment.
- Private backend communication.
- Load balancer health checks enabled.

Architecture Layers

Presentation Layer

The presentation layer is responsible for interacting with users.

Components:

- Application Load Balancer
- Frontend EC2 Instances
- Web Interface

Functions:

- Handles client requests.
- Displays application content.
- Forwards API requests to backend services.

Application Layer

The application layer contains business logic.

Components:

- Internal Load Balancer
- Backend EC2 Instances

Functions:

- Processes user requests.
- Performs validation.
- Executes application logic.
- Communicates with database.

Database Layer

The database layer stores application data.

Components:

- Amazon RDS MySQL
- Database Subnet Group

Functions:

- Data storage.
- Data retrieval.
- Backup and recovery.
- Multi-AZ failover support.

Advantages of Three-Tier Architecture

Scalability

Each tier can scale independently based on demand.

Security

Backend and database servers remain inaccessible from the internet.

High Availability

Multi-AZ deployment reduces downtime.

Performance

Load balancing distributes traffic across multiple servers.

Maintainability

Changes can be made to one tier without affecting others.

AWS Services and Their Roles:

Amazon VPC

Provides network isolation and control.

Subnets

Separate public and private resources.

Internet Gateway

Enables internet access.

NAT Gateway

Allows private instances to access the internet securely.

Application Load Balancer

Distributes incoming traffic.

EC2

Hosts frontend and backend applications.

Auto Scaling Group

Automatically adjusts server count.

Amazon RDS

Managed relational database service.

Security Groups

Acts as virtual firewalls.

Security Design

- Public access only through ALB.
- Frontend servers remain in private subnets.
- Backend servers remain in private subnets.

- Database remains in private database subnets.
- Security groups restrict access between tiers.

High Availability Design

Resources are deployed across three Availability Zones:

- AZ-A
- AZ-B
- AZ-C

Benefits:

- Fault tolerance
- Automatic failover
- Improved reliability

Project Outcome

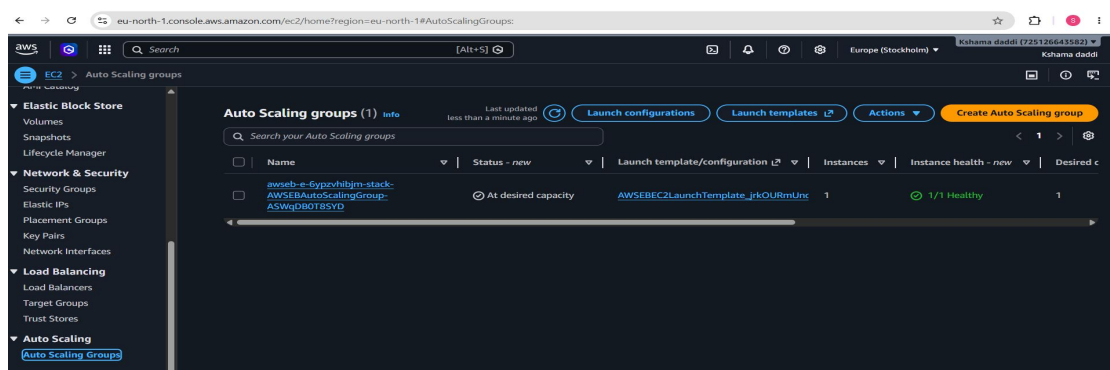
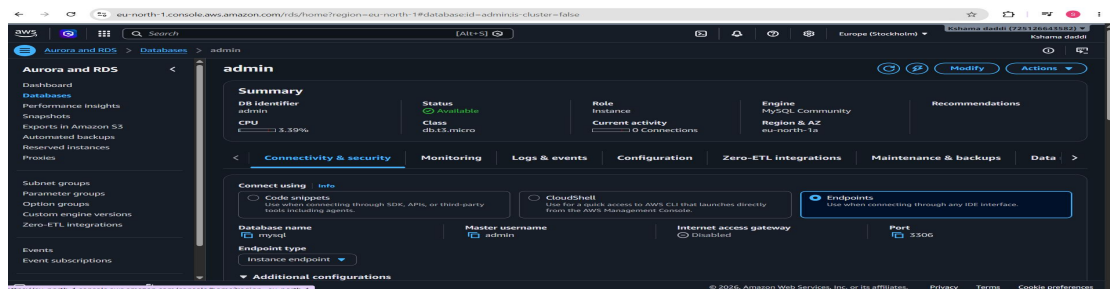
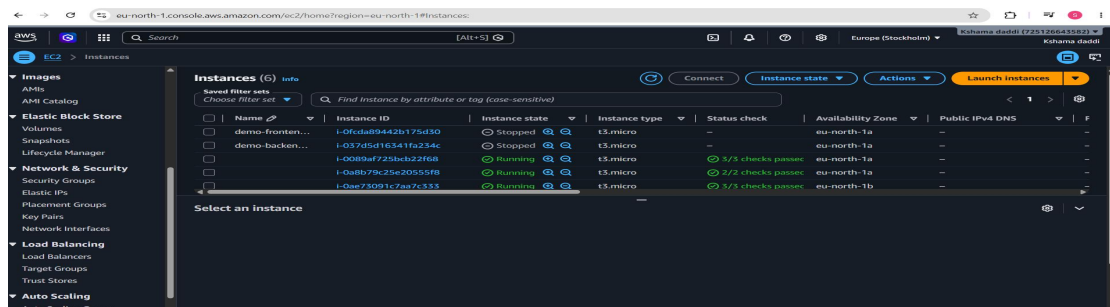
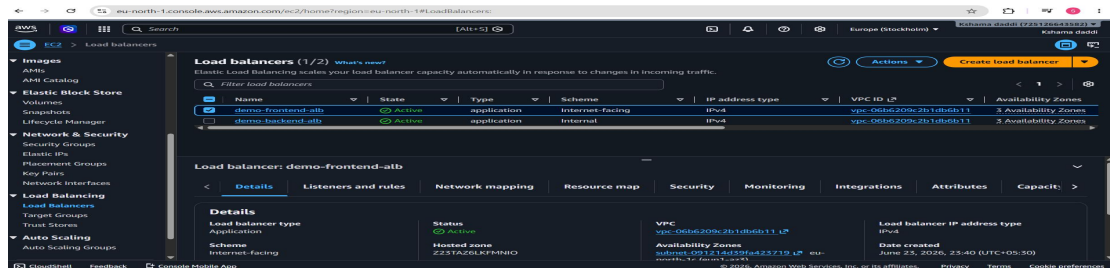
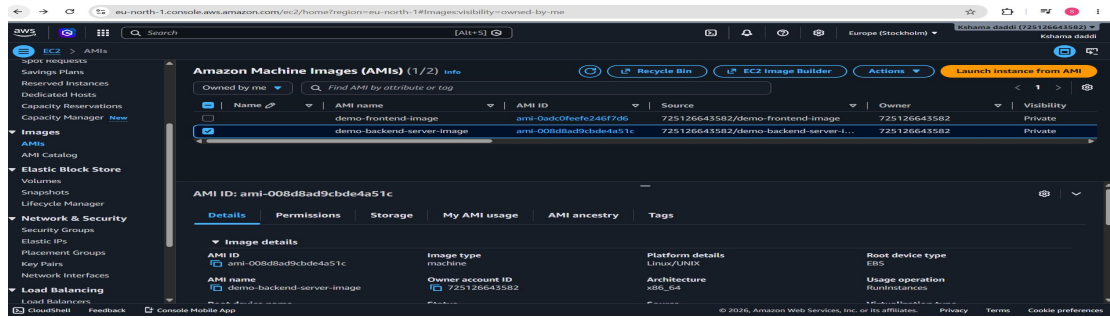
Successfully implemented a scalable and highly available Three-Tier Architecture on AWS using:

- Multi-AZ Deployment
- Application Load Balancers
- Auto Scaling Groups
- Private Networking
- Amazon RDS MySQL
- Secure Security Groups

Conclusion

The AWS Three-Tier Architecture provides a secure, scalable, and highly available environment for hosting enterprise applications. By separating presentation, application, and database layers across multiple Availability Zones and integrating AWS managed services, organizations can improve performance, security, and operational efficiency.

Screenshot:



```
eu-north-1.console.aws.amazon.com/ec2-instance-connect:ssh/home?addressfamily=ipv4&connType=standard&instanceId=i-0fcd89442b175d308&osUser=ec2-user&region=eu-north-1&ss...

Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-core-2.50.1-1.amzn2023.0.1.x86_64
Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.noarch
Verifying : perl-error-1.0.17030-2.amzn2023.0.1.noarch
Verifying : perl-file-find-1.27-477.amzn2023.0.9.noarch
Verifying : perl-git-2.50.1-1.amzn2023.0.1.noarch
Verifying : perl-TermReadKey-2.38-9.amzn2023.0.9.x86_64
Verifying : perl-lib-0.65-477.amzn2023.0.9.x86_64

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64
perl-error-1.0.17030-2.amzn2023.0.1.noarch
perl-file-find-1.27-477.amzn2023.0.9.noarch
perl-git-2.50.1-1.amzn2023.0.1.noarch
perl-lib-0.65-477.amzn2023.0.9.x86_64
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64

Complete!
ec2-user@ip-10-0-4-7 ~$ sudo vi /etc/nginx/nginx.conf
ec2-user@ip-10-0-4-7 ~$ ls
ec2-user@ip-10-0-4-7 ~$ find /etc/nginx/nginx.conf
bash: find: command not found
ec2-user@ip-10-0-4-7 ~$ find /etc/nginx/nginx.conf
/etc/nginx/nginx.conf
ec2-user@ip-10-0-4-7 ~$ cd /etc/nginx/nginx.conf
bash: cd /etc/nginx/nginx.conf: Not a directory
ec2-user@ip-10-0-4-7 ~$ sudo vi /etc/nginx/nginx.conf
ec2-user@ip-10-0-4-7 ~$ sudo cat /etc/nginx/nginx.conf
cat metadata expiration check: 0:17:41 ago on Thu Jun 25 06:31:27 2026.
Package git-2.50.1-1.amzn2023.0.1.x86_64 is already installed.
dependencies resolved.
Nothing to do.
Complete!
ec2-user@ip-10-0-4-7 ~$

i-0fcd89442b175d30 (demo-frontent-instance)
PublicIP: 51.20.252.28 PrivateIP: 10.0.4.7
CloudShell Feedback Console Mottle App
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences
```

```
eu-north-1.console.aws.amazon.com/ec2-instance-connect:ssh/home?region=eu-north-1&connType=standard&instanceId=i-037d5d16a34f1a234c&osUser=ec2-user&sshPort=22&addressFa...

server version: 8.4.8 Source distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql [(none)]> CREATE DATABASE IF NOT EXISTS hello_world;
Query OK, 1 row affected (0.027 sec)

mysql [(none)]> USE hello_world;
Database changed
mysql [(none)]>
mysql [(none)]> -- Create messages table
mysql [(none)]> CREATE TABLE IF NOT EXISTS messages (
->   id INT AUTO INCREMENT PRIMARY KEY,
->   message VARCHAR(255) NOT NULL,
->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-> );
Query OK, 0 rows affected (0.064 sec)

mysql [(none)]>
mysql [(none)]> -- Insert some initial data
mysql [(none)]> INSERT INTO messages (message) VALUES
-> ('Hello from the database!'),
-> ('Welcome to our three-tier architecture demo'),
-> ('This is a simple example showing frontend, backend, and database');
Query OK, 3 rows affected (0.035 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Not secure demo-frontent-alb-530571243.eu-north-1.elb.amazonaws.com

Tech with Ajit

Basic Three-Tier Architecture Example

Message from Database:

- hello this is 3-tire architecture (2026-06-25 07:49:51)
- Hello from the database! (2026-06-25 07:04:03)
- Welcome to our three-tier architecture demo (2026-06-25 07:04:03)
- This is a simple example showing frontend, backend, and database (2026-06-25 07:04:03)

Add New Message

New Message:

Save Message